Data Engineering Best Practices

This document serves the purpose of defining the accepted and expected data engineering best practices.

Target Audience: This guide is intended for all data engineers and anyone looking to build their data pipeline using the China Data Platform and Data Factory.

> *"Every data pipeline starts with understanding its source and ends with ensuring it meets business needs. This model ensures alignment across technical design and stakeholder expectations."*

Item	Description	Remark
Document owner	Data Foundation & Engineering Lead & Engineering Excellence Manager	
First publish	May 15, 2025	

Versions

Version Number	Description	Creator / Modifier	Update Date
1.0	Add overall mental model and hands-on practices of building data pipelines	@Leon Ding	May 15, 2025
1.1	Structure documentation	@Yue3 Zhu	May 30, 2025
1.2	Added Data Cleansing & Transformation techniques based on Spark SQL for Part II	@Yue3 Zhu	Jun 9, 2025
1.3	Modify the best practice by embedding some of items in RDA	@Lei Ren	Jun 11, 2025

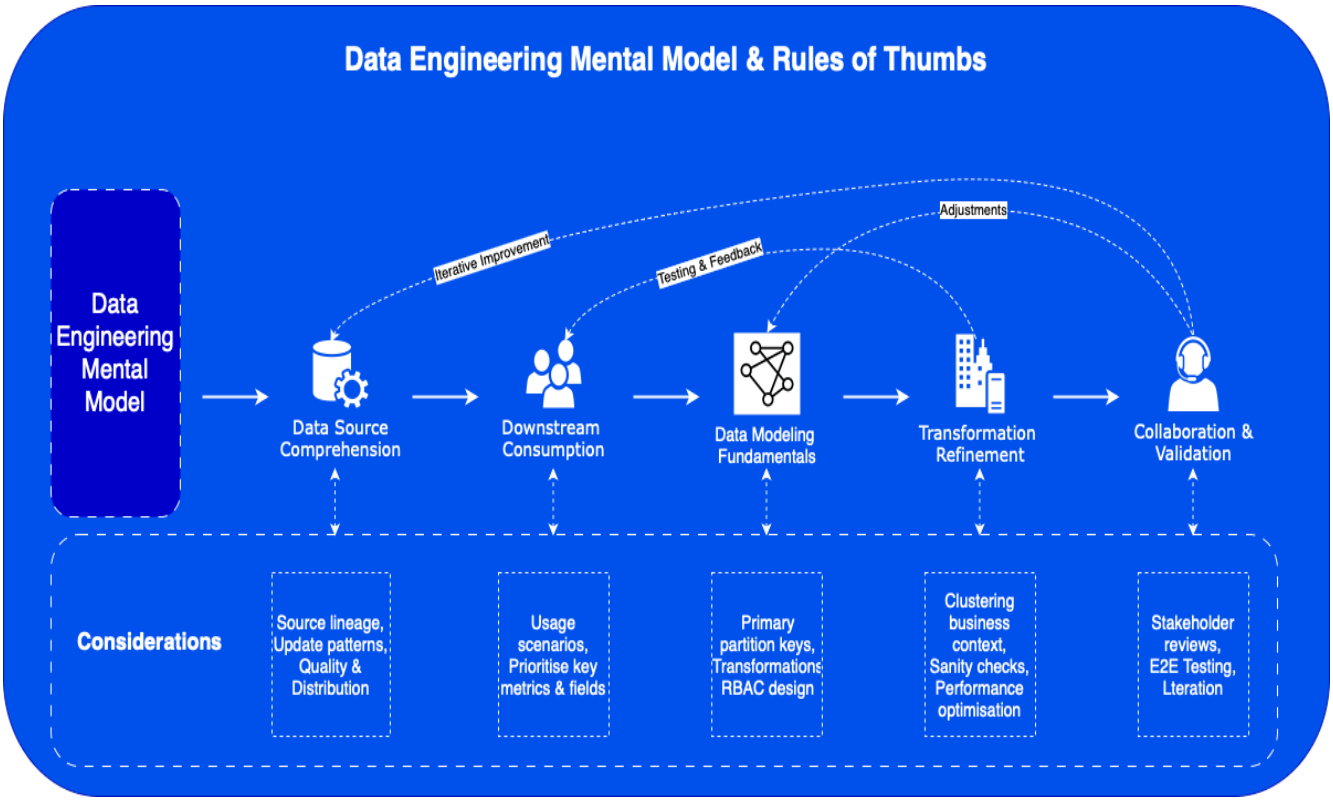
	standards. Add RDA standards as a separate references		
1.4	Added examples to each “Hands-on“ part	@Yue3 Zhu	Jul 14, 2025
1.5	Added guidance on E2E Data Reconciliation	@Andy Yuan	Jul 14, 2025

Glossary of terms and abbreviations

Terms	Abbreviations
TTD	Talk-to-Data
LOV	List of Values
UDF	User-Defined Function
DDL	Data Definition Language
GBU	Global Business Unit
RDA	Reusable Data Asset

---

1. Recommended Data Engineering Mental Model and Rule of Thumbs



Data Source Comprehension

Objective: Establish a foundational understanding of data origins and characteristics.

- Understand your **data source**
  - Where does such data come from in the first place (its ancestor)?
  - What is the update frequency and mechanism (batch vs stream)
  - What is the data model and structure
  - Do you understand the real meaning of each column? Are the names good enough and non-confusing
  - How is the data quality overall
  - What is the distribution of each field
  - How many data tables do I need to build our pipeline?

A sample of data sources for Sanofi China Data Platform are:

Application	APM ID	Description	Business Domain
Content Library	APM0069995	A centralized content composing and publishing center for China	Develop Content and Material Approval

Refer to Applications List in APM Portal to understand more about the source.

Downstream Consumption Mapping

Objective: Align data outputs with business needs through end-user requirements.

- Understand how your ***Product*** (dataset) will be consumed in the downstream
  - Typical usage scenario
  - Typical analytics model
    - common dimensions
    - hierarchies and measures
- Look at the downstream analysis scenarios
  - How does a visual in a dashboard looks like
  - How does a pivot table analyzing such domain data looks like
  - What are the typical questions users will ask using ***TTD***?
- Decide what is the high priority columns vs less prioritized ones in your final dataset, work on the priority ones first and guarantee their quality

Data Modeling Fundamentals

Objective: Design robust schemas for efficient storage and querying.

> *Data modeling is foundational for efficient data engineering. A poorly designed model leads to performance bottlenecks, unreliable insights, and increased maintenance overhead. On the other hand, a well-structured model empowers organizations to make data-driven decisions with confidence.*

- Design your primary key and unique columns!
- Design your partition keys by learning the underlying data warehouse and data lake!
- Imagine your dataset will go through a series of transformative steps, including:
  - Column wise normalization
    - Fill `na` and standardize `N/A` values
    - Standardize LOV
    - Think twice on how `0, 1, Y, N`, etc shall be used
  - Single row changes
    - Use mapping logic from another mapping dataset and bring out the mapped out value (ALWAYS CHECKING THE PRIMARY KEY / JOIN KEYS IN YOUR MAPPING TABLES, ESPECIALLY IF THEY COME FROM OTHER TEAMS)
    - Calculate new values based on existing, e.g. unit price from sales and volume, map out a new flag based on existing ones
    - Add new columns or overwrite existing ones (recalculate)
  - Row split
    - Performance co-own (duplication), adding sum/count flag
    - To split a record according to an allocation rule, should the original record be removed?
  - Row merge
    - When aggregating (grouping) by specific fields, consider whether you will retain the detailed dataset following the aggregation process.
  - Add control fields (timestamp, change/modify/new flags, access control & role flags)

- A combination of above (but try to avoid complex UDF, use simple transformation language as much as you can for readability, maintainability, and efficiency)

Transformation Process Refinement

> **Objective**: Optimize workflow readability, maintainability, and performance.

- Organize your transformation rules into clusters based on the underlying business logic.
  - e.g. if you want to standardize `year`, `month`, `day`, weekly, group all the transformation together
- Document each group using non-technical language
  - e.g. “based on the standard brand name, bring out the BU and GBU fields based on the standard product hierarchy” rather than “join `dwd.fact_xx` with the `ads.product` on the brand key…”)
- Do a sanity check after each group of transformations
  - compare row count before vs after see if this is expected
  - use your common sense (e.g. do we need mm:ss for a daily-based transaction?)
  - Examine the data distribution for any unusual values.
  - check if hierarchies follow one parent — multiple children or not, go back to fix the mapping if needed
- Test your overall pipeline for errors, consistencies, conflicts, and performance (run time)
  - understand how many rows will be impacted from each run (# of rows deleted, # of rows inserted/upserted)
- Chaining your pipeline together to form a more complicated transformation
  - Repetitive statements like “`SELECT x, y, z FROM temp table`” can significantly hinder the readability and maintainability of your SQL code for humans.
  - To enhance performance, aim to minimize the use of temporary tables whenever possible and focus on performing more computations in-memory.
  - Separate your Data Definition Language (DDL) from your transformation logic.

Collaboration & Validation Protocols

> **Objective**: Ensure alignment through cross-functional review and rigorous testing.

- **Stakeholder Engagement**
  - Joint reviews with data stewards/requestors to validate logic
  - Rapid failure iteration and adjustments (following SOP for dataset releases)
- **End-to-End Testing**
  - Pipeline validation for errors, consistency, and conflicts
  - Performance profiling (runtime metrics, row impact analysis)

E2E Data Reconciliation

> **Objective**: Ensures **data consistency, accuracy, and completeness** between sources and data assets

The reconciliation control points should be set across the data pipeline:

- **Data ingestion phase**
  - for dimensional data, perform full data load at agreed frequency;
  - for incremental data extraction scenarios, the system is typically configured to extract data from the past several days with some data overlapping (dedup in later stage);
  - for some sources with histories of data issues, a full set of primary key data is also extracted to perform a full comparison in later stage, ensuring consistency with the source data.
- **Data loading phase**
  - Tencent COS MD5 checksum mechanism implemented to ensure data ingested in BDMP are transferred as-is into our PSA (COS buckets).
  - If a full set of primary key data is extracted earlier, it will be used for a full data comparison and sync up between data staged in PSA and our Bronze. For other cases, we use *upsert* based on PK to guarantee data completeness and avoid duplications.
- **Data transformation phase**
  - perform various checking points on data reconciliation between Bronze and Gold, e.g.
    - basic row count check
    - consistency check of measure fields
    - aggregated value checks
    - sample check of records
- **Data presentation phase**
  - additional layer of controls can be set to ensure data consistency across different data consumers, e.g. compare key metrics across different Analytical Apps (Plai, OneCI, OneCI.Mobile, IBP, etc.)

---

2. Hands-on Best Practices on Building Data Pipelines

Below info includes Sanofi China's Data Lake platform, products and concepts (RDA)

Some common fields that you probably want to add to all RDAs

- `source_system` - predefined, standardized source system name, ie. ONEMDM_CN, LUCI, as your gold layer dataset may contains same nature of data from different source systems
- `year`, `quarter`, `month`, (week) - standard time frames columns to make downstream analysis and data chunking easier. These are usually calculated based on the transaction date or snapshot date, depending on the nature of your dataset
- most frequently used dimension hierarchy, ie. `product`, `customer`, `employee` hierarchy. Yes, this is duplicative, but it is to ease downstream usage. Make it a habit.
- incremental flags - can be `timestamp` or monotonically increment cursor value, this is more for downstream data incremental load.

UPDATED

> *This practice is suggested to apply in "Flat" Data Model, or OneDataset. But not recommended in commercial Business Intelligence products, eg Microsoft Power BI, the explanation is here (power-bi-star-schema-or-single-table).*
>
> *Fields in RDA could be categorized as below:*

Field Category	Field	Definition	Example
Date/Time MANDA...	Periodical Fields	The data snapshot date is essential for updating full load / or incremental load by certain period datasets.	Optional, add `year`, `quarter`, `month`, `week`, or any combinations like `year-quarter`, `year-week` for downstream system better use.
Date/Time MANDA...	Transaction Date	The transaction occurrence date and time are ideal for incremental data loading.	INTERACTION `interaction_date` EVENT `event_plan_date` `event_actual_date` INTERACTION `survey_date`
Dimensional MANDA...	Geography / Product / Customer Fields	Entities have attributes which describe specific properties.	HCO `country_cd` `province_cd` `city_cd` `county_cd` INTERACTION `gbu` `bu` `brand` `hco_etms_cd` `hcp_etms_cd`
Transaction MANDA...	Sales Fields	Contains the numeric columns use to summarize and aggregate measure values	IMS_CV `actual_volume` `ly_actual_volume`
Technical OPTION...	Metadata	Metadata for OneDataset	Optional, add `source_system` `eim_last_modified_date`

> *These metadata is required for the Bronze layer when data is first loaded into Data Lake, refer to* Data development *for details.*

Time period

- Create time period columns like `year`, `quarter`, `month`, `week`, `weekday`, `day_of_year` per necessary based on not just the transaction date but also some flagging dates (`list_date`, `change_effective_date`)
- If the company strictly follows a fiscal calendar, use the fiscal calendar to generate `fiscal_year`, `fiscal_quarter`, `fiscal_month`, etc. While if there is no such restrictions, use ISO calendar
- Week - use ISO week number if possible, unless the company defines its own week count rule. `week_num` should be starting from 1 and ends at 52 or 53, and shall not start from 1 at the beginning of each quarter. If you do need

a `week_num` reset for each quarter, create a new column called `quarter_week_num`

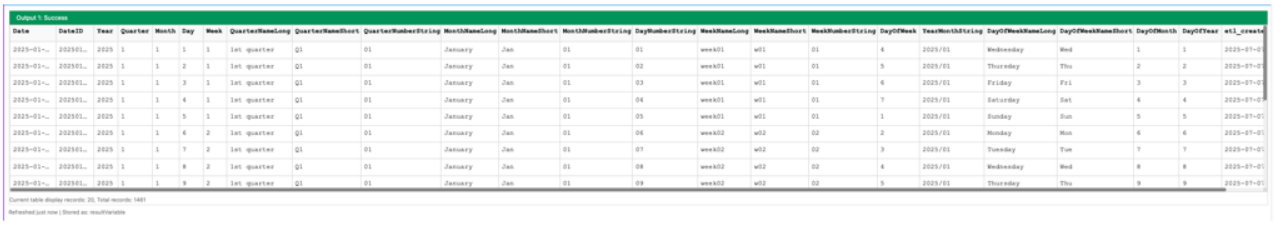
- Not all datasets need a datetime field containing hour/minute/seconds. If you dataset is to be consumed by downstream only at a finest date granularity, just truncate the hh/mm/ss coming from your source systems...

UPDATED

> *Periodical information could be generated based on any date columns from the source system, depending on consumer's needs, it could be extended to any form of date data, including* `year`, `quarter`, `month`, `week`, `year-month`, `year-quarter`, `year-quarter-month`, `year-ISO week number`, *and any abbreviations and truncations part of the date data.*
>
> *A generic date table or dimension is crucial for most of analytical use cases, eg, OneDataset.*
>
> *You can create it anywhere in your ETL process, but the overall recommendation is:* ***to create it not too far downstream, and to provide it centrally for the whole organization****.*
>
> *Also, recreating such a date dimension for any of your reports, datasets, marts or other layers just wastes resources.*
>
> *Below scripts in PySpark that you can reference a date table, can just fetch the columns and rows OneDataset need from that table and they are good to go.*

Examples

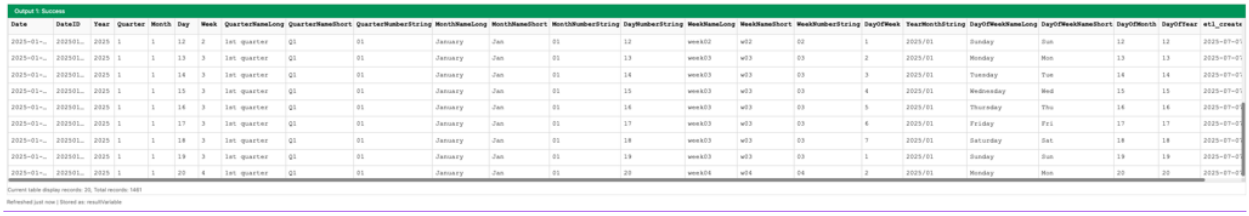
The goal is to create a calendar table as below:



PySpark Implementation for standard calendar table - 1st Approach

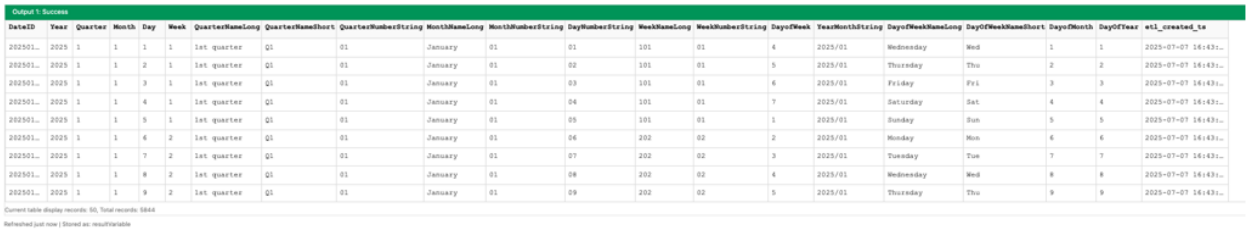
```
# Load packages
from pyspark.sql import functions as F
from datetime import datetime, timedelta

# Define boundaries
startdate = datetime.strptime('2025-01-01','%Y-%m-%d')
enddate   = (datetime.now() + timedelta(days=365 * 3)).replace(month=12,
day=31)  # datetime.strptime('2025-10-01','%Y-%m-%d')

# Define column names and its transformation rules on the Date column
column_rule_df = spark.createDataFrame([
    ("DateID",              "cast(date_format(date, 'yyyyMMdd') as int)"),
# 20250101
    ("Year",                "year(date)"),
# 2025
    ("Quarter",             "quarter(date)"),
# 1
    ("Month",               "month(date)"),
# 1
    ("Day",                 "day(date)"),
# 1
```

```
    ("Week",                "weekofyear(date)"),
# 1
    ("QuarterNameLong",     "date_format(date, 'QQQQ')"),
# 1st qaurter
    ("QuarterNameShort",    "date_format(date, 'QQQ')"),
# Q1
    ("QuarterNumberString", "date_format(date, 'QQ')"),
# 01
    ("MonthNameLong",       "date_format(date, 'MMMM')"),
# January
    ("MonthNameShort",      "date_format(date, 'MMM')"),
# Jan
    ("MonthNumberString",   "date_format(date, 'MM')"),
# 01
    ("DayNumberString",     "date_format(date, 'dd')"),
# 01
    ("WeekNameLong",        "concat('week', lpad(weekofyear(date), 2, '0'))"),
# week 01
    ("WeekNameShort",       "concat('w', lpad(weekofyear(date), 2, '0'))"),
# w01
    ("WeekNumberString",    "lpad(weekofyear(date), 2, '0')"),
# 01
    ("DayOfWeek",           "dayofweek(date)"),
# 1
    ("YearMonthString",     "date_format(date, 'yyyy/MM')"),
# 2025/01
    ("DayOfWeekNameLong",   "date_format(date, 'EEEE')"),
# Sunday
    ("DayOfWeekNameShort",  "date_format(date, 'EEE')"),
# Sun
    ("DayOfMonth",          "cast(date_format(date, 'd') as int)"),
# 1
    ("DayOfYear",           "cast(date_format(date, 'D') as int)"),
# 1
# Sonntag
], ["new_column_name", "expression"])

# Explode dates between the defined boundaries into one column
start = int(startdate.timestamp())
stop  = int(enddate.timestamp())
df = spark.range(start, stop,
60*60*24).select(col("id").cast("timestamp").cast("date").alias("Date"))

# Loops over all rules defined in column_rule_df adding the new columns
for row in column_rule_df.collect():
    new_column_name = row["new_column_name"]
    expression = expr(row["expression"])
    df = df.withColumn(new_column_name, expression)

# Final Calendar Dataframe
df = df.withColumn("etl_created_ts",  F.date_format(current_timestamp(), 'yyyy-
MM-dd HH:mm:ss')) \

df.show(50)
```



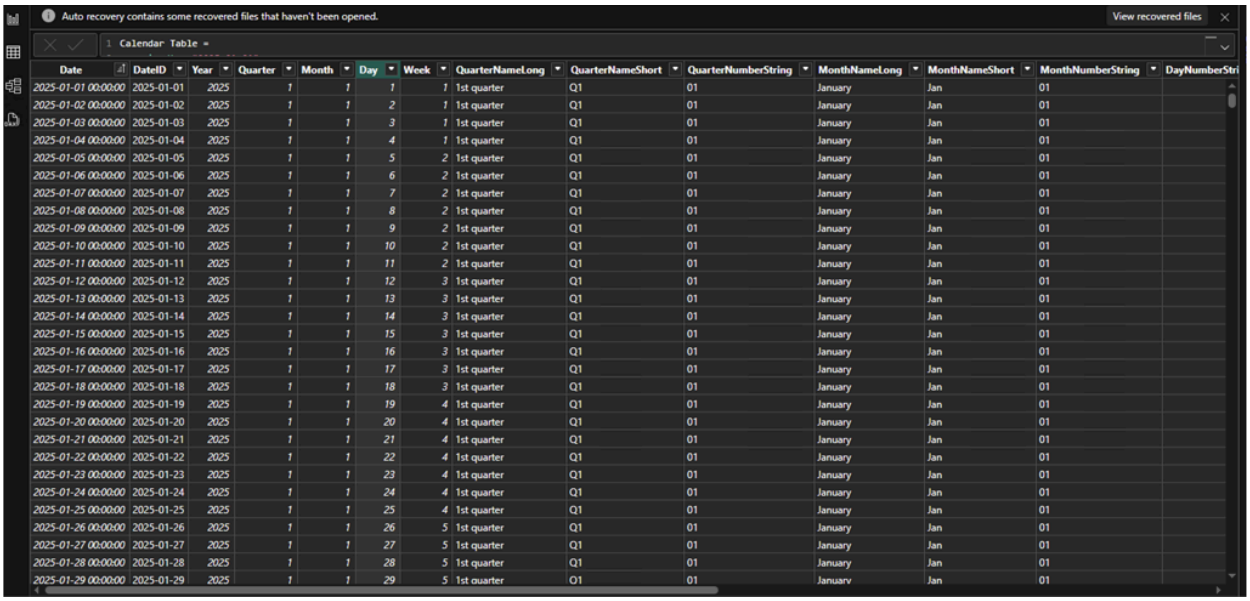
PySpark Implementation for standard calendar table - 2nd Approach

```python
# 初始化dataframe
calendarDf = spark.createDataFrame([(1,)], ["id"])

# 增加自定义列
calendarDf = calendarDf \
.withColumn("Date",F.explode(F.expr("sequence(to_timestamp('2025-01-01 00:00:00'), to_timestamp('2040-12-31 00:00:00'), interval 1 day)"))) \
.withColumn("DateID", F.date_format('Date', 'yyyyMMdd')) \
.withColumn("Year", F.year("Date").cast("string")) \
.withColumn("Quarter", F.quarter('Date')) \
.withColumn("Month", F.month('Date')) \
.withColumn("Day", F.dayofyear('Date')) \
.withColumn("Week", F.weekofyear('Date')) \
.withColumn("QuarterNameLong", F.date_format('Date', 'QQQQ')) \
.withColumn("QuarterNameShort", F.date_format('Date', 'QQQ')) \
.withColumn("QuarterNumberString", F.date_format('Date', 'QQ')) \
.withColumn("MonthNameLong",  F.date_format('Date', 'MMMM')) \
.withColumn("MonthNumberString", F.date_format('Date', 'MM')) \
.withColumn("DayNumberString",   F.date_format('Date', 'dd')) \
.withColumn("WeekNameLong",  F.concat('Week', F.lpad(F.weekofyear('Date'), 2, '0'))) \
.withColumn("WeekNameShort",  F.concat('W', F.lpad(F.weekofyear('Date'), 2, '0'))) \
.withColumn("WeekNumberString",  F.lpad(F.weekofyear('Date'), 2, '0')) \
.withColumn("DayofWeek", F.dayofweek('Date')) \
.withColumn("YearMonthString", F.date_format('Date', 'yyyy/MM')) \
.withColumn("DayofWeekNameLong", F.date_format('Date', 'EEEE')) \
.withColumn("DayOfWeekNameShort", F.date_format('Date', 'EEE')) \
.withColumn("DayofMonth", F.date_format('Date', 'd').cast('int')) \
.withColumn("DayOfYear", F.date_format('Date', 'D').cast('int'))

# 删除指定列
calendarDf = calendarDf.drop("id", "Date", "date")

# 对 DataFrame 去重
calendarDf = calendarDf.distinct()

# 增加metadata
calendarDf = calendarDf.withColumn('etl_created_ts', F.date_format(F.current_timestamp(), 'yyyy-MM-dd HH:mm:ss'))

calendarDf = calendarDf.orderBy("DateID")

calendarDf.show(50)
```



Microsoft PowerBI for a standard calendar table

```
calendar =
var min_dt = MIN('fact'[date])                                          -- define the minimum date boundaries from a transacation date
var max_dt = MAX('fact'[date])                                          -- define the maximum date boundaries from a transacation date
return ADDCOLUMNS(
    CALENDAR(min_dt, max_dt),
    "DateID", FORMAT([Date], "yyyy-mm-dd"),
    "Year", YEAR([Date]),
    "Quarter", QUARTER([Date]),
    "Month", MONTH([Date]),
    "Day", DAY([Date]),
    "Week", WEEKNUM([Date]),
    "QuarterNameLong", SWITCH(QUARTER([Date]), 1, "1st quarter", 2, "2nd quarter", 3, "3rd quarter", 4, "4th quarter"),
    "QuarterNameShort", "Q" & QUARTER([Date]),
    "QuarterNumberString", "0" & QUARTER([Date]),
    "MonthNameLong", FORMAT([Date], "mmmm"),
    "MonthNameShort", FORMAT([Date], "mmm"),
    "MonthNumberString", FORMAT([Date], "mm"),
    "DayNumberString", DATEDIFF(DATE(YEAR([Date]),1,1), [Date], DAY) + 1,
    "WeekNameLong", "Week" & " " & IF(LEN(WEEKNUM([Date])) = 1, "0" & WEEKNUM([Date]), WEEKNUM([Date])),
    "WeekNameShort", "W" & IF(LEN(WEEKNUM([Date])) = 1, "0" & WEEKNUM([Date]), WEEKNUM([Date])),
    "DayofWeek", WEEKDAY([Date]),
    "YearMonthString", FORMAT([Date], "yyyy/mm"),
    "DayofWeekNameLong", FORMAT([Date], "dddd"),
    "DayofWeekNameShort", FORMAT([Date], "ddd"),
    "DayofMonth", DAY([Date]),
    "DayofYear", DATEDIFF(DATE(YEAR([Date]),1,1), [Date], DAY) + 1
)
```



Date	DateID	Year	Quarter	Month	Day	Week	QuarterNameLong	QuarterNameShort	QuarterNumberString	MonthNameLong	MonthNameShort	MonthNumberString	DayNumberStri
2025-01-01 00:00:00	2025-01-01	2025	1	1	1	1	1st quarter	Q1	01	January	Jan	01	
2025-01-02 00:00:00	2025-01-02	2025	1	1	2	1	1st quarter	Q1	01	January	Jan	01	
2025-01-03 00:00:00	2025-01-03	2025	1	1	3	1	1st quarter	Q1	01	January	Jan	01	
2025-01-04 00:00:00	2025-01-04	2025	1	1	4	1	1st quarter	Q1	01	January	Jan	01	
2025-01-05 00:00:00	2025-01-05	2025	1	1	5	2	1st quarter	Q1	01	January	Jan	01	
2025-01-06 00:00:00	2025-01-06	2025	1	1	6	2	1st quarter	Q1	01	January	Jan	01	
2025-01-07 00:00:00	2025-01-07	2025	1	1	7	2	1st quarter	Q1	01	January	Jan	01	
2025-01-08 00:00:00	2025-01-08	2025	1	1	8	2	1st quarter	Q1	01	January	Jan	01	
2025-01-09 00:00:00	2025-01-09	2025	1	1	9	2	1st quarter	Q1	01	January	Jan	01	
2025-01-10 00:00:00	2025-01-10	2025	1	1	10	2	1st quarter	Q1	01	January	Jan	01	
2025-01-11 00:00:00	2025-01-11	2025	1	1	11	2	1st quarter	Q1	01	January	Jan	01	
2025-01-12 00:00:00	2025-01-12	2025	1	1	12	3	1st quarter	Q1	01	January	Jan	01	
2025-01-13 00:00:00	2025-01-13	2025	1	1	13	3	1st quarter	Q1	01	January	Jan	01	
2025-01-14 00:00:00	2025-01-14	2025	1	1	14	3	1st quarter	Q1	01	January	Jan	01	
2025-01-15 00:00:00	2025-01-15	2025	1	1	15	3	1st quarter	Q1	01	January	Jan	01	
2025-01-16 00:00:00	2025-01-16	2025	1	1	16	3	1st quarter	Q1	01	January	Jan	01	
2025-01-17 00:00:00	2025-01-17	2025	1	1	17	3	1st quarter	Q1	01	January	Jan	01	
2025-01-18 00:00:00	2025-01-18	2025	1	1	18	3	1st quarter	Q1	01	January	Jan	01	
2025-01-19 00:00:00	2025-01-19	2025	1	1	19	4	1st quarter	Q1	01	January	Jan	01	
2025-01-20 00:00:00	2025-01-20	2025	1	1	20	4	1st quarter	Q1	01	January	Jan	01	
2025-01-21 00:00:00	2025-01-21	2025	1	1	21	4	1st quarter	Q1	01	January	Jan	01	
2025-01-22 00:00:00	2025-01-22	2025	1	1	22	4	1st quarter	Q1	01	January	Jan	01	
2025-01-23 00:00:00	2025-01-23	2025	1	1	23	4	1st quarter	Q1	01	January	Jan	01	
2025-01-24 00:00:00	2025-01-24	2025	1	1	24	4	1st quarter	Q1	01	January	Jan	01	
2025-01-25 00:00:00	2025-01-25	2025	1	1	25	4	1st quarter	Q1	01	January	Jan	01	
2025-01-26 00:00:00	2025-01-26	2025	1	1	26	5	1st quarter	Q1	01	January	Jan	01	
2025-01-27 00:00:00	2025-01-27	2025	1	1	27	5	1st quarter	Q1	01	January	Jan	01	
2025-01-28 00:00:00	2025-01-28	2025	1	1	28	5	1st quarter	Q1	01	January	Jan	01	
2025-01-29 00:00:00	2025-01-29	2025	1	1	29	5	1st quarter	Q1	01	January	Jan	01	

Choose the right column type

- Choose the type of the column (string, date, datetime, decimal, int) that best suits the purpose according to common sense rather than some technical decisions...
- Do you want a column for date or datetime? Do you really need the time part?
- Be frugal on the string/varchar types, do not always use String or VARCHAR(255/1024) for all string fields, do you really need that length of characters? For category field containing A/B/C as values, consider to use CHAR(1) or Category type if the SQL engine supports it
- For integer column types, consider to use int4, int8 to save space and also to limit the range as an intrinsic control on the quality. For instance, if you have a field capturing the weekday_num, consider use a small int to limit a wrong weekday_num of 100...

> *Data types in Spark v4.0 for reference*

UPDATED

Data Profiling on OneDataset - Demand to Cash - IMS_CV

#	Columns	As-Is Type	Sample	Maximum Value	Optimized Type
1	year	varchar	2025	4	varchar(4)
2	quarter	varchar	Q1	2	varchar(2)
3	month	varchar	1	2	varchar(2)
4	yearmonth	varchar	202501	6	varchar(6)
5	sales_date	date	23/1/2025	30/6/2025	date

6	workday_of_month	integer	16	23	integer
7	gbu	varchar	GenMed	14	varchar(50)
8	bu	varchar	Whale	14	varchar(50)
9	sub_bu	varchar	Whale	23	varchar(50)
10	franchise	varchar	WHL_NEU	23	varchar(50)
11	area	varchar	WHL_NEU	20	varchar(50)
12	ptgroup_cd	varchar	WHL_NEU	9	varchar(50)
13	ptgroup_nm	varchar	鲸鱼神经产品线	14	varchar(50)
14	brand	varchar	Depakine	14	varchar(50)
15	disease	varchar		4	varchar(50)
16	product	varchar	Depakine Tab	25	varchar(50)
17	package_cd	varchar	13	8	varchar(20)
18	package	varchar	Depakine 500mg SR Tab BT1x30	38	varchar(50)
19	gmid	varchar			varchar(50)
20	dosage	decimal(20,6)	3	16.68	decimal(10,6)
...	...	...	...	...	...

DDL of OneDataset - Demand to Cash - IMS_CV - Before

```
-- iceberg.rda_demand_to_cash.ims_cv definition

-- Drop table

```

DDL of OneDataset - Demand to Cash - IMS_CV - After

```
-- iceberg.rda_demand_to_cash.ims_cv definition

-- Drop table

```

```sql
-- DROP TABLE iceberg.rda_demand_to_cash.ims_cv;

CREATE TABLE iceberg.rda_demand_to_cash.ims_cv (
    "year" varchar,
    quarter varchar,
    "month" varchar,
    yearmonth varchar,
    sales_date date,
    workday_of_month integer,
    gbu varchar,
    bu varchar,
    sub_bu varchar,
    franchise varchar,
    area varchar,
    ptgroup_cd varchar,
    ptgroup_nm varchar,
    brand varchar,
    disease varchar,
    product varchar,
    package_cd varchar,
    package varchar,
    gmid varchar,
    dosage decimal(20,6),
    -- 字段过多，省略...
);
```

```sql
-- DROP TABLE iceberg.rda_demand_to_cash.ims_cv;

CREATE TABLE iceberg.rda_demand_to_cash.ims_cv (
    "year" varchar(4),
    quarter varchar(2),
    "month" varchar(2),
    yearmonth varchar(6),
    sales_date date,
    workday_of_month integer,
    gbu varchar(50),
    bu varchar(50),
    sub_bu varchar(50),
    franchise varchar(50),
    area varchar(50),
    ptgroup_cd varchar(50),
    ptgroup_nm varchar(50),
    brand varchar(50),
    disease varchar(50),
    product varchar(50),
    package_cd varchar(20),
    package varchar(50),
    gmid varchar(50),
    dosage decimal(10,6),
    -- 字段过多，省略...
);
```

Normalize the list of values

- Review the list of values in category fields, see if there are any similar but different values, with difference in the ways of (lower case vs upper case, whitespace vs no space, - vs _, English vs Chinese, typo vs correct spelling). Standardize the list of values.
- If your stakeholders tell you to create “bespoke” values to be identified for special downstream purposes, challenge them and yourself whether you just follow the “guideline”? Or you should create dedicated columns for that differentiation purpose and convince them not to build customized values into a category type field...
- For flag columns especially binary ones, whether we use 1/0, Y/N, y/n depends on your org’s choice. Using 1/0 is more performance friendly but more business challenging...
- Similarly, for n/a, null, empty, invalid values, standardize the convention you use within not only your dataset but also across datasets.
- Use your judgment and experience to determine which fields are the ones you should standard the list of values. Usually they are hierarchical dimension fields, fields with names of flags, codes, types, etc.

UPDATED

> Refer to RDA Modeling Standardization | 4 Field Value Standardization for standardise the list of values. @Lei Ren please check if below rules are correct.

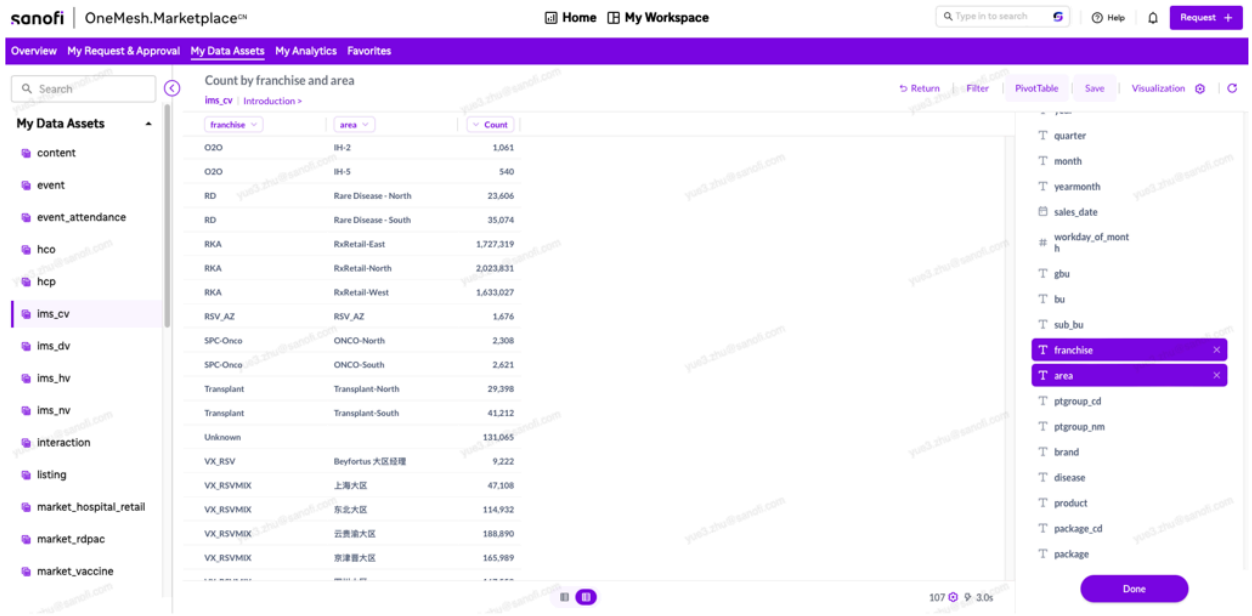
In RDA, we have below standards for list of values and naming conventions:

1. If the information has standard value from source systems, eg: OneMDM, use it consistently.

2. If it does not have standard value, UPPER case all the values in dimensional info, such as `COMPANY NAME`
3. Use Chinese value OR English value, not mixing both.
4. Words separators, use `“-“` consistently, instead of one space `“ “` or underscore symbol `“_“`.
5. Handle missing value, such as `NA`, `N/A`, `Null`, `Empty`, in RDA we leave it as is, default will use `Empty`.

<details>
<summary>A "Good" Example</summary>

[Questions: Can you find the examples which violate “Normalize the list of values“ rule?]



</details>

Choose a meaningful and interpretable user defined value

- Say “**NO**” to your stakeholder if they propose to use `aaa`, `abc`, `suffix-1,2,3` as a special value to be added to your dataset to cut out a dedicated piece of the data to [“reflect the business fact”]. Instead, ask for what does these `1,2,3`, `aaa`, `abc` means actually. Say it out and implement the fundamental reasons into your pipeline.
  - For instance, if a suffix 1 means a special type of category as an exception to a standard value, why not just create another “standard” value if that is so important? Or keep the standard but create another column to flag out this “special” logic with meaningful column name like sub_..., is_special_...

UPDATED

> In the realm of business, understanding the distinction between business context and technical language is crucial. It is essential to establish a clear definition or criteria to assess whether the "User defined value" resonates with both stakeholders. This alignment ensures effective communication and mutual understanding between business and technical teams.

In RDA, IMS_CV data asset, we have mutiple `sales_channel` and `sales_sub_channel` in below table:

“ `Difference` “ here is a good example of interpretable user defined value, proposed by GTMC I&A team for `“AB Sell out - In Market Sales“`.

sales_channel	sales_sub_channel	count
Hospital Channel	Hospital	552,466
Hospital Channel	POV	314,676
Retail Channel	DTP-DCP	38
Retail Channel	**Difference**	2,847
Retail Channel	HMP	66
Retail Channel	Pharmacy	132

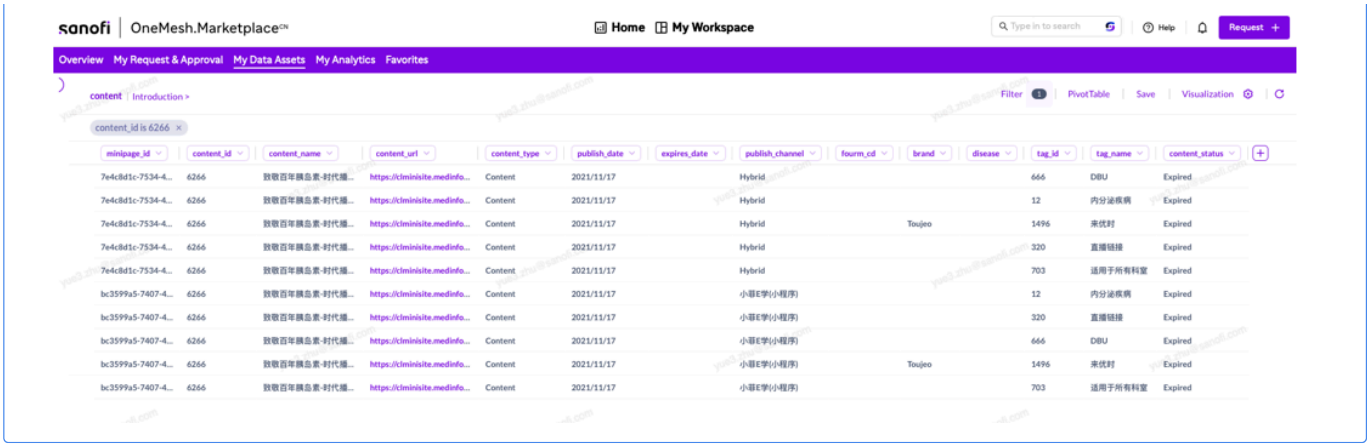
What is the right level of granularity

- As our RDA is in many situations denormalized, we do expect a certain level of duplication (on the row axis) in our dataset, but once you decide to expand/explode the rows, ask yourself if there a way to easily identify the
  a. unique count of the occurrences
  b. the main copy of the record
  c. how to avoid the double count of measures
- When you do expand the rows, check if the columns that should expand across different rows are properly and completed propagated into the duplicated / expanded rows. For example, you design your dataset at a granularity @ content * tag (debatable whether this is a best design), have you duplicated the brand field into all the expanded rows, or you only assigned brand to one of the row?
- When should I put multi-values as a list into one row and when should I explode the different values into different rows? There is no simple answer for that. But here are some principles to help you decide:
  - Are the multi-values frequently used in downstream? if so, consider to split into multiple rows;
  - What does frequent use mean? if one downstream sometimes use that field for a algorithm modeling purpose rather than as a slicer to be used in a dashboard, then it's not a "frequent" usage scenario.

UPDATED

> In OneMesh.Marketplace, we expand the rows for data asset "Content", since each content will have more Tags (tag_id, tag_name) etc..
>
> Such tag names, like "DBU", "内分泌疾病"，"来优时" could be easily used as a Dashboard Slicer (eg, OneCI Dashboard).

Content with Tags



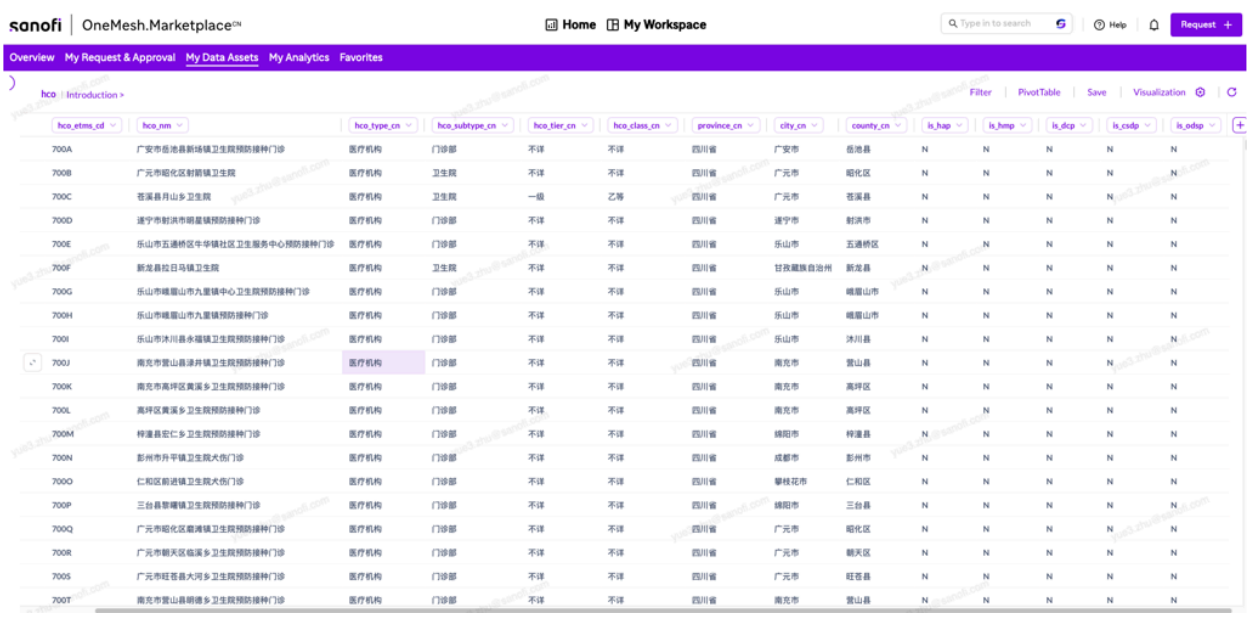
Create new columns

- **Do not hesitate to create/add flags to indicate business logics** - some records in the in_market_sales dataset is in-market sales, while others are not, create a flag called is_ims and assign yes or no;
  - For example, in employee table, in order to differentiate field force from non field force, rather than educate your data customers that you need to use field `A == 'xx'` AND field `B == 'y'` to judge, just create a new column called `is_ff` and assign boolean values
- **A column is a column and should be used for only one purpose**. Do not mix up different stuff and put it into one column and request your customers to learn and memorize rules to dissect that column and translate further into usable data.
  - For example, you have a column from your data source capturing a `y/n` flag and a date if the status is `y`. It makes much more sense to create two columns in your pipeline and breakdown the original column and assign the different parts of the information into the two columns, one for y/n flag, one for date.
- **Consider add columns to concat / combine multiple columns**.
  - For example, downstream constantly needs to look at a reporting line chain from AD to Rep, considering creating a column by concatenating these different names into one field. Or you have 3 flags and your downstream uses a business rule to filter out some data based on a combination of these 3 flags, why not creating a new flag column with a meaningful and straightforward name capturing the combined logic?

UPDATED

> This section share part of the reason from Data Engineering Best Practices | Normalize the list of values

Data asset - HCO, HCP



Data Asset - HCO

Complex “business logic”

- You might encounter so called “complex business logic” where you need to use multiple, recursive if-else logic gates to determine the final result. This is inevitable at the end of the day. Do **DOCUMENT** this human-made complex logic.
- A few ways to implement this:
  - 1) Hard-code such rules as layered CASE WHEN using SQL
  - 2) Use pySpark and richer python language to simplify the conditional judgment, for more complicated scenarios, consider using UDF
  - 3) Define a mapping table and capture the multiple to 1 logic and use joins to bring the result. There is no simple answer to which one is the best.
- If you adopt approach 1) and 2) as above, also think about how you are going to maintain the rules going forward, at what frequency? How do you make sure the requestor will inform you when the rules change promptly? How do you avoid the rules being repeated implemented in different pipelines and creating gaps in the rules?
- However, if your transform logic pipeline is full of hard-coded complex logics, ask yourself:
  - Should I build a mapping table and ask the requestor to maintain it
  - Do I have to implement these so called complex business logics all in RDA?

UPDATED

> The term “complex business logic” requires clarification. Is it referring to a specific business context, or is it more about a technical approach to implementations?

A "Good" Example

```sql
-- ads_ci.svw_fact_ice_ecard_attribute source

CREATE OR REPLACE
VIEW `svw_fact_ice_ecard_attribute` (`id`,
`gbu`,
`bu`,
`fourthmcode`,
`fourm_name`,
`fourm_adop`,
`title`,
`code`,
`brands`,
`tp`,
`pp`,
`keymessage`,
`minisitecontpageid`,
`categories`,
`content_status`,
`isdeleted`,
`created_dt`,
`modified_dt`,
`created_by`,
`modified_by`,
`job_id`) AS WITH `fourm_name` (`fourm_name`,
`fourm_cd`,
`adoption_ladder`) AS (
SELECT
    '重度AD局部治疗控制不佳' AS `fourm_name`,
    'MAT-CN-2509452' AS `fourm_cd`,
    '知晓者' AS `adoption_ladder`
UNION ALL
SELECT
    '重度AD局部治疗控制不佳' AS `fourm_name`,
    'MAT-CN-2415373' AS `fourm_cd`,
    '知晓者' AS `adoption_ladder`
UNION ALL
SELECT
    '重度AD传统系统治疗控制不佳' AS `fourm_name`,
    'MAT-CN-2509453' AS `fourm_cd`,
    '试用者' AS `adoption_ladder`
UNION ALL
SELECT
    '重度AD传统系统治疗控制不佳' AS `fourm_name`,
    'MAT-CN-2415375' AS `fourm_cd`,
    '试用者' AS `adoption_ladder`
UNION ALL
SELECT
    'BSA>5%' AS `fourm_name`,
    'MAT-CN-2509454' AS `fourm_cd`,
    '使用者 – 倡导者' AS `adoption_ladder`
```

Hard code for a dimensional data instead of a mapping table

ads_ci.dim_evnt_master.sql

```sql
    END                                 AS  event_subcat,
    event_create_dt,
    event_start_dt,
    event_end_dt,
    owner_emp_cd,
    owner_pos_cd,
    CASE
        WHEN event_chan='OFFLINE' THEN 0
        WHEN event_chan='ONLINE' THEN 1
        WHEN event_chan='HYBRID' THEN 2
        ELSE NVL(event_chan,0)
    END                                  AS  event_chan,
    CASE
        WHEN t1.source_system_id='1033' THEN 'Vaccine'
        WHEN t3.employee_cd ='W50135930' THEN 'Vaccine'
        WHEN SUBSTRING(t1.event_bu_cd,1,1) ='V' THEN 'Vaccine'
        WHEN t3.lvl0_pos_cd IS NOT NULL THEN t3.sta_bu_nm                                   --REP BU
        WHEN SUBSTRING(event_client_cd,2,2)='P2' THEN event_bu_cd
        WHEN t3.lvl0_pos_cd IS NULL AND t3.employee_cd IS NOT NULL AND t3.position_cd1 IS NOT NULL THEN coalesce(t5.bu_name_en,event_bu_cd)          --非REP，优先级1
        WHEN t3.lvl0_pos_cd IS NULL AND t3.position_cd1 IS NULL AND t3.employee_cd IS NOT NULL AND t3.position_cd2 IS NOT NULL THEN coalesce(t6.bu_name_en,event_bu_cd)       --非REP，优先级2
        ELSE coalesce(event_bu_cd,SUBSTRING(event_client_cd,2,2))
     END                                 AS  bu_cd,
    event_primary_org_unit_cd,
    event_primary_org_unit_nm,
    event_compl_dt,
    event_loc_nm,
    event_loc_state_nm,
    event_loc_city_nm,
    event_loc_addr,
    COALESCE(t1.event_auto_nm,t2.event_auto_nm) AS event_auto_nm,
    event_first_req_dt,
    event_aprv_step,
    event_pending_aprv_dt,
    event_aprv_status,
    event_aprv_emp_cd,
    event_aprv_comments,
    is_digital,
    event_bud_amt,
    event_mkt_project_cd,
    status_cd,
    source_system_id,
    source_system_cd,
    is_hard_del,
    eim_created_dt,
    eim_last_modified_dt,
    event_delegator_cd,
    owner_lm_taken_by,
    mc_taken_by,
    finance_taken_by,
    year_month
FROM dws_ci.flat_event t1
LEFT JOIN sub_event_auto_nm t2
```

Hard code of a special employee code, if-else

Column orders do matter

- Do not randomly put columns here and there. Whenever you want to create new columns, always find the “best” place to put them
- Put the common and indexing columns always at the very beginning (time period, etc.)
- For hierarchical mapping columns, choose and align with team a norm to order these hierarchical columns in descending or ascending order (e.g. package-brand/product-BU-GBU, or in the opposite order)
- Group related columns together. For example, put all product related columns together, then group all customer related columns, etc.
- Grouping flag columns together and with the related columns. For instance, you can place the is_new_launch, is_vbp, etc. flags together with product related dimensions.
- Group measure columns (quantity, amount) together, and put that group in later positions

UPDATED

> Refer to RDA Modeling Standardization | 3 Field Order Standardization column orders.

Join/merge/map

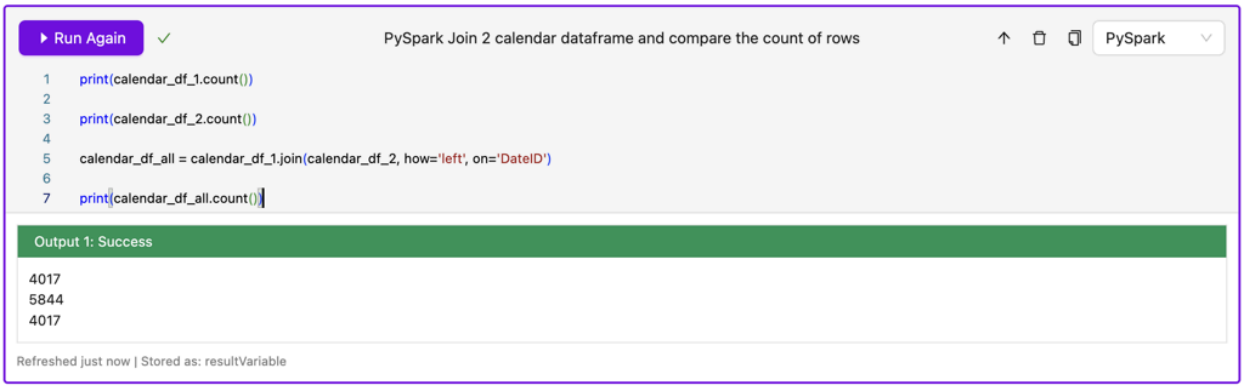
- Most joins (or merges in the context of pandas dataframe) is used to bring additional fields out into your pipeline
- In some cases, you will drop the join key and only keep the joined result. Do not be afraid to drop certain fields from your bronze/silver layer tables
- But do check if your mapping table has a proper UNIQUE, PRIMARY KEY. If you create duplicated rows in your mapping dataset, you are inviting troubles...
- Always compare the total number of rows before and after the join. They should always be the same (except the case described below for explicit row split/duplicate)

UPDATED

> If you are using Pandas, chose “merge“ method, and use “join“ method if you are using PySpark.

Easy way to compare row count from PySpark for dataframes

```
print(calendar_df_1.count())

print(calendar_df_2.count())

calendar_df_all = calendar_df_1.join(calendar_df_2, how='left', on='DateID')

print(calendar_df_all.count())
```



How to duplicate or split records into multiple rows

- If you are using SQL or pyspark, decide your split / duplicate logic, left join (SQL JOIN or df.join) with a right table with multiple rows given one join key. Right table structure typically is like:
  - row1: join_key_A | mapping_A or split_ratio_A
  - row2: join_key_A | mapping_B or split_ratio_B
- Or you can use pySpark and write your own inner loop: 1) duplicate a row by hand, 2) alter certain fields in the duplicated row, 3) append the duplicated rows to the Dataframe

UPDATED

> Consider carefully before duplicating a record in your dataset; does it align with the requirements of the downstream system? Will it create confusion for your business?

Use union and unionByName to merge two DataFrame

```
# Get specific date as new DataFrame
calendar_jan_1 = calendar_jan.filter(F.col("DayofWeekNameShort") == "Wed")

calendar_jan_1.show(1)

calendar_jan_2 = calendar_jan.filter(F.col("DayofWeekNameShort") == "Wed").withColumn("QuarterNameLong", F.upper(F.col("QuarterNameLong")))

calendar_jan_2.show(1)

# Check if columns match
calendar_jan_1.printSchema()
calendar_jan_2.printSchema()

# Merge two DataFrame with union
calendar_jan_union_1 = calendar_jan_1.union(calendar_jan_2)

# Merge two DataFrame with unionByName
calendar_jan_union_2 = calendar_jan_1.unionByName(calendar_jan_2)

calendar_jan_union_1.show(20)

calendar_jan_union_2.show(20)
```

Output 4: Success

DateID	Date	Year	Quarter	Month	Day	Week	QuarterNameLong	QuarterNameShort	QuarterNumberString	MonthNameLong	MonthNumberString	DayNumberString	WeekNumberString	DayofWeek	YearMonthString	DayofWeekNam
202501…	2025-01-…	2025	1	1	1	1	1st quarter	Q1	01	January	01	01	01	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	8	2	1st quarter	Q1	01	January	01	08	02	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	15	3	1st quarter	Q1	01	January	01	15	03	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	22	4	1st quarter	Q1	01	January	01	22	04	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	29	5	1st quarter	Q1	01	January	01	29	05	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	1	1	1ST QUARTER	Q1	01	January	01	01	01	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	8	2	1ST QUARTER	Q1	01	January	01	08	02	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	15	3	1ST QUARTER	Q1	01	January	01	15	03	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	22	4	1ST QUARTER	Q1	01	January	01	22	04	4	2025/01	Wednesday

Current table display records: 10, Total records: 10

Output 5: Success

DateID	Date	Year	Quarter	Month	Day	Week	QuarterNameLong	QuarterNameShort	QuarterNumberString	MonthNameLong	MonthNumberString	DayNumberString	WeekNumberString	DayofWeek	YearMonthString	DayofWeekNam
202501…	2025-01-…	2025	1	1	8	2	1st quarter	Q1	01	January	01	08	02	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	15	3	1st quarter	Q1	01	January	01	15	03	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	22	4	1st quarter	Q1	01	January	01	22	04	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	29	5	1st quarter	Q1	01	January	01	29	05	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	1	1	1ST QUARTER	Q1	01	January	01	01	01	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	8	2	1ST QUARTER	Q1	01	January	01	08	02	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	15	3	1ST QUARTER	Q1	01	January	01	15	03	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	22	4	1ST QUARTER	Q1	01	January	01	22	04	4	2025/01	Wednesday
202501…	2025-01-…	2025	1	1	29	5	1ST QUARTER	Q1	01	January	01	29	05	4	2025/01	Wednesday

Make measure fields business meaningful

- Choose the right unit of measure to make the downstream consumption meaningful.
  - For instance, when you measure **duration of an event**, it doesn't make sense to use millisecond... When a measure field is for capture quantity, ask yourself do you have another column to explain what is the unit of measurement of the quantity - is it package, unit, box, etc?
  - For measures like quantity, amount, etc. Choose the right numeric type - not everything is a `float` or `double`, choose the right `decimal` type. Or if the underlying SQL engine does not support Decimal (fixed precision float numbers), apply `round()` function before writing your result back to the data lake/warehouse
- For percentage and ratio columns, use `decimal` type as well with fixed precision as well.
  - For instance, if you calculate the `market_share%` and add it into your dataset, define the type as `decimal(4)` and example values be 0.1234 so that when presenting such values in reports, it is shown as `12.34%`.

UPDATED

> This section pertains to a segment of Data Engineering Best Practices | Choose the right column type. Occasionally, it is necessary to convert Type A to Type B to enhance the clarity and readability of the information for data consumers, particularly from a business perspective.

Why cannot a sales target data type be set to Integer?

```
SELECT
    yearmonth,
    gbu,
    bu,
    franchise,
    brand,
    product,
    package,
    actual_volume,
```

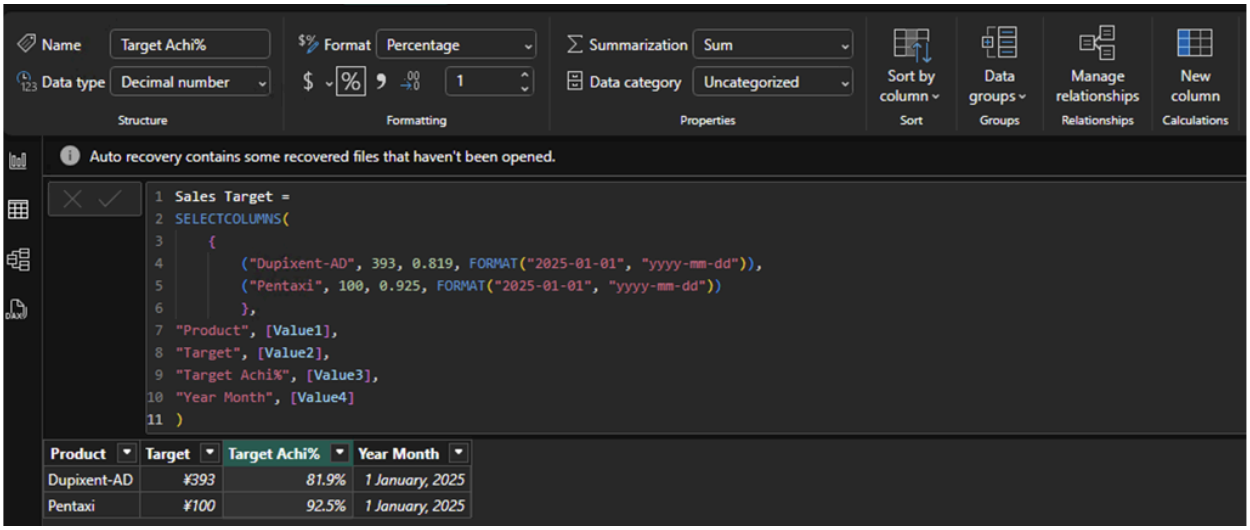
```
    target_volume,
    mr_position_cd
FROM
    iceberg_catalog1.rda_demand_to_cash.ims_cv
WHERE
    yearmonth = '202501'
    AND
franchise <> 'NON-TP'
    AND target_volume > 0
    AND brand = 'Dupixent'
    AND product = 'Dupixent-AD'
LIMIT 100
```

yearmonth	gbu	bu	franchise	brand	product	package	actual_volume	target_volume	mr_position_cd
202501	Specialty Care	SPC-Dupi	Dupi-AD	Dupixent	Dupixent-AD	Dupixent 300MG/2ML INJ PS2_AD	0	78.6	POS_75809036
202501	Specialty Care	SPC-Dupi	Dupi-AD	Dupixent	Dupixent-AD	Dupixent 300MG/2ML INJ PS2_AD	0	78.6	POS_75849988
202501	Specialty Care	SPC-Dupi	Dupi-AD	Dupixent	Dupixent-AD	Dupixent 300MG/2ML INJ PS2_AD	0	78.6	POS_75775933
202501	Specialty Care	SPC-Dupi	Dupi-AD	Dupixent	Dupixent-AD	Dupixent 300MG/2ML INJ PS2_AD	0	78.6	POS_75849987
202501	Specialty Care	SPC-Dupi	Dupi-AD	Dupixent	Dupixent-AD	Dupixent 300MG/2ML INJ PS2_AD	0	78.6	POS_75826037

Dupixent-AD Target Volume in 2025 Jan

How does Microsoft Power BI deal with the meaningful measures

```
Sales Target =
SELECTCOLUMNS(
    {
        ("Dupixent-AD", 393, 0.819, FORMAT("2025-01-01", "yyyy-mm-dd")),
        ("Pentaxi", 100, 0.925, FORMAT("2025-01-01", "yyyy-mm-dd"))
        },
"Product",      [Value1],
"Target",       [Value2],
"Target Achi%", [Value3],
"Year Month",   [Value4]
)
```



Microsoft Power BI Deal with Measures

Add calculated measures

- Use column calculations in SQL or pySpark. Do not perform calculation using for loop to iterate over rows in a UDF with pySpark. The transform speed will be much worse

- Most calculated fields will only need +, -, *, / conditioning on certain control flags. Do not overcomplicate it.

UPDATED

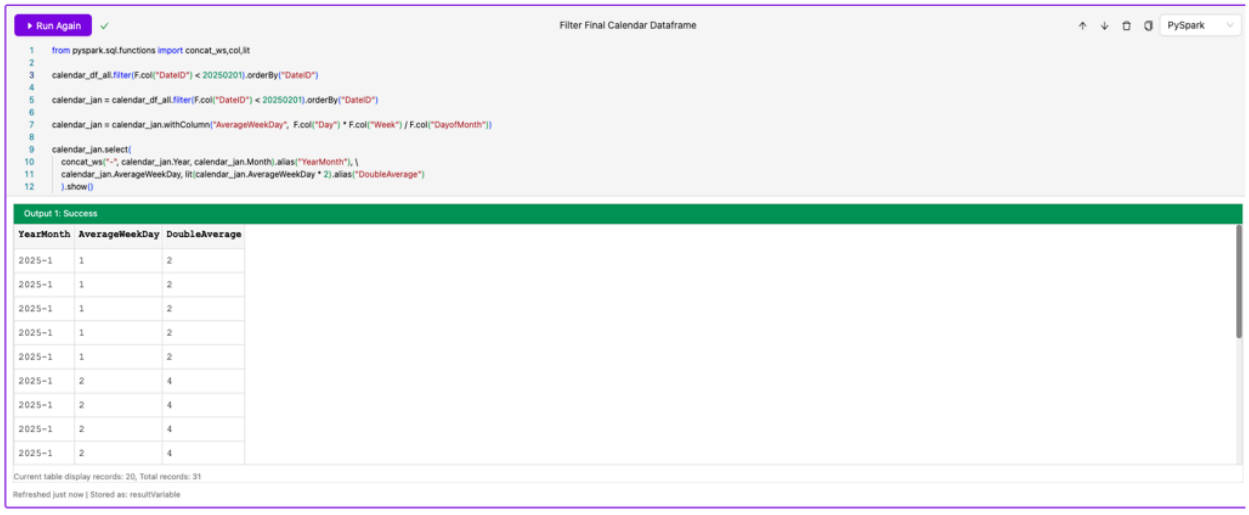
> ⚠ PySpark DataFrames are designed for distributed data processing, so direct row-wise iteration should be avoided when working with large datasets. Instead, consider using Spark's built-in transformations and actions to process data more efficiently.

Calculated measures with PySpark

> Mostly for simple computations, instead of iterating through using `map()` or `foreach()`, you should use either `DataFrame.select()` or `DataFrame.withColumn()` in conjunction with PySpark SQL functions.
>
> Below is an example of using `select()` and `withColumn()`.

Calculated measures with PySpark

```
from pyspark.sql.functions import concat_ws,col,lit

# Filter Dataframe
calendar_df_all.filter(F.col("DateID") < 20250201).orderBy("DateID")

calendar_jan = calendar_df_all.filter(F.col("DateID") < 20250201).orderBy("DateID")

calendar_jan = calendar_jan.withColumn("AverageWeekDay",  F.col("Day") * F.col("Week") / F.col("DayofMonth"))

calendar_jan.select(
    concat_ws("-", calendar_jan.Year, calendar_jan.Month).alias("YearMonth"), \
    calendar_jan.AverageWeekDay, lit(calendar_jan.AverageWeekDay * 2).alias("DoubleAverage")
    ).show()
```

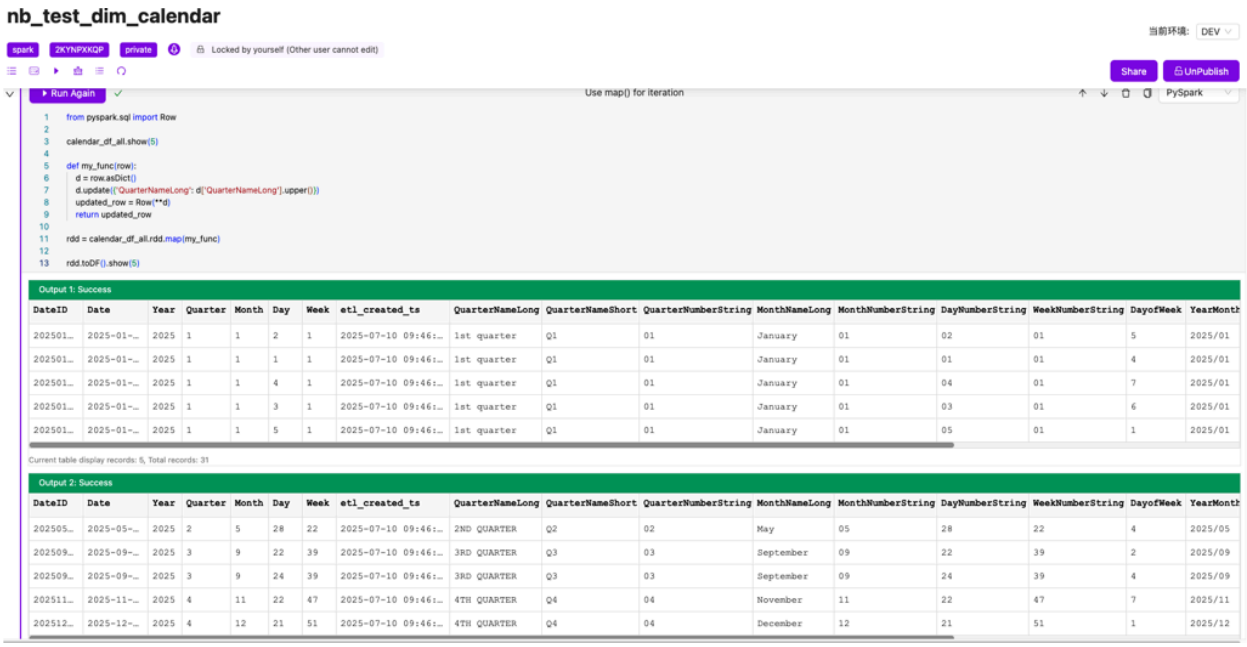


Loop / iteration with PySpark map()

> PySpark `map()` Transformation is used to loop/iterate through the PySpark DataFrame/RDD by applying the transformation function (lambda) on every element (Rows and Columns) of RDD/DataFrame.
>
> PySpark doesn't have a `map()` in DataFrame instead it's in RDD hence we need to convert DataFrame to RDD first and then use the `map()`. It returns an RDD and you should Convert RDD to PySpark DataFrame if needed.

Approach 1: Define functions for iteration with map()

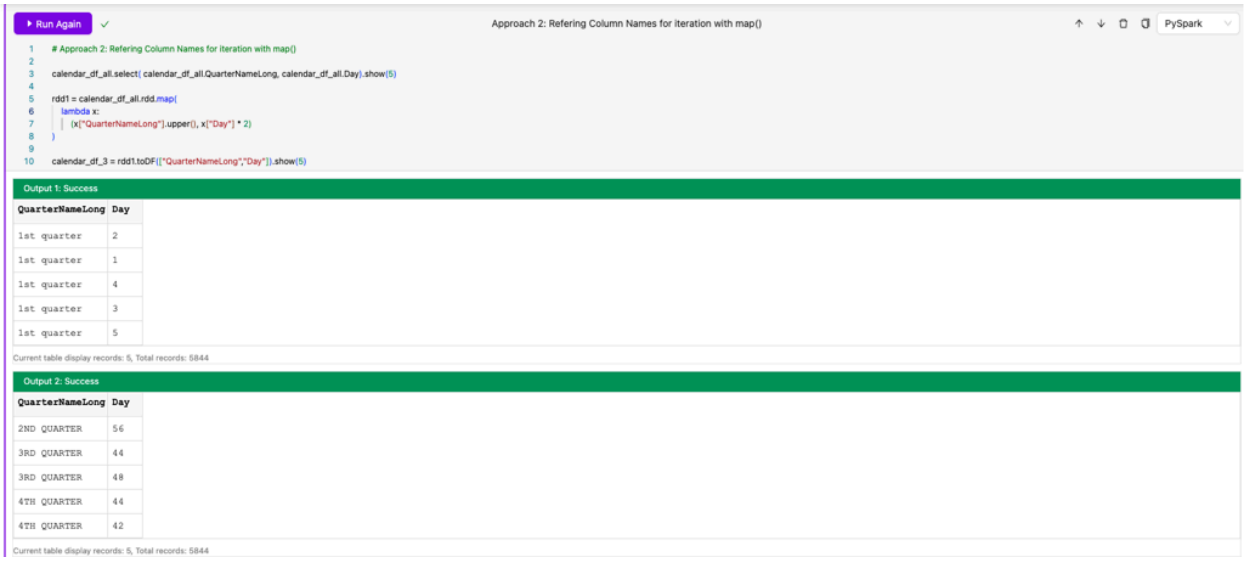
```python
from pyspark.sql import Row

calendar_df_all.show(5)

# Approach 1: Define functions for iteration with map()
def my_func(row):
    d = row.asDict()
    d.update({'QuarterNameLong': d['QuarterNameLong'].upper()})
    d.update({'MonthNameLong': d['MonthNameLong'].lower()})
    updated_row = Row(**d)
    return updated_row

rdd = calendar_df_all.rdd.map(
    lambda row:
        my_func(row)
    )

rdd.toDF(["QuarterNameLong", "MonthNameLong"]).show(5)
```



Approach 2: Referring Column Names for iteration with map()

```python
# Approach 2: Referring Column Names for iteration with map()

calendar_df_all.select( calendar_df_all.QuarterNameLong,
calendar_df_all.Day).show(5)

```

```
rdd1 = calendar_df_all.rdd.map(
    lambda x:
        (x["QuarterNameLong"].upper(), x["Day"] * 2)
)

calendar_df_3 = rdd1.toDF(["QuarterNameLong","Day"]).show(5)
```



Reference

Documentation	URL
Mathematical Functions in PySpark 4.0.0	Functions — PySpark 4.0.0 documentation
PySpark 3.5 Tutorial For Beginners with Examples	PySpark 3.5 Tutorial For Beginners with Examples
Working with DataFrames in PySpark	Working with DataFrames in PySpark – Dataquest